Generative Gaussian Splatting for Unbounded 3D City Generation

Anonymous Author
Anonymous Institution

AI-Generated

Abstract

Generating photorealistic, unbounded 3D city scenes is critical for applications in gaming, autonomous driving simulation, and virtual reality. Existing NeRF-based city generation methods, while capable of producing high-quality imagery, suffer from prohibitively slow rendering speeds and heavy memory requirements. Although 3D Gaussian Splatting (3D-GS) enables real-time rendering, naively scaling it to city-level scenes demands billions of Gaussian primitives and excessive VRAM, rendering it infeasible. In this paper, we present ***GaussianCity****, the first generative framework that leverages 3D-GS for unbounded city-scale scene generation. The key to our approach is a compact, visibility-aware* BEV-Point *representation that keeps GPU memory consumption constant as the scene grows, achieved by extruding BEV height and semantic maps into 3D point clouds and pruning invisible points via ray-intersection visibility and a binary density map. To recover fine structural details from these sparse, unordered point sets, we design a spatial-aware* BEV-Point Decoder *comprising a Point Serializer and a Point Transformer (PTv3) backbone, conditioned on per-instance style codes via modulated MLPs that predict Gaussian RGB attributes. Extensive experiments on GoogleEarth (drone-view) and KITTI-360 (street-view) demonstrate that GaussianCity achieves state-of-the-art generation quality while running at 93 ms per frame—more than 60× faster than the best NeRF-based competitor—with only 1.39 GB of VRAM.*

1. Introduction

Synthesizing high-fidelity, large-scale 3D city environments has broad applications in gaming, film production, autonomous driving simulation, and virtual reality. A generative model capable of producing unlimited, photorealistic city content on-the-fly would significantly reduce the manual effort currently required for 3D content creation. Despite growing interest, generating unbounded 3D cities that are simultaneously geometrically coherent, visually realistic, and computationally efficient remains a major open challenge.

Recent efforts in 3D scene generation have largely relied on Neural Radiance Fields (NeRF) [17] as the underlying scene representation. Methods such as SceneDreamer [7] and CityDreamer [24] adopt compositional NeRF-based architectures to decompose city scenes into semantic elements like buildings, roads, and vegetation, achieving reasonable image quality. However, NeRF's implicit volumetric integration is inherently slow at inference time: for instance, CityDreamer requires over 5,500 ms to render a single frame, far from real-time requirements. More broadly, NeRF-based generative models struggle to scale to unbounded outdoor scenes without carefully engineered scene parameterizations [24].

3D Gaussian Splatting (3D-GS) [13] has recently emerged as an efficient alternative scene representation that achieves real-time rendering via differentiable rasterization of explicit 3D Gaussian primitives. Its explicit nature and fast rendering pipeline make it attractive for generative modeling. However, deploying 3D-GS for large-scale city generation introduces a fundamental scalability challenge: a naive BEV-to-Gaussian lifting approach produces tens of millions of points for a city-scale scene, immediately causing GPU out-of-memory (OOM) errors even on high-end hardware.

In this work, we introduce **GaussianCity**, the first generative framework built on 3D-GS for photorealistic, unbounded 3D city generation. Our design is motivated by two key insights. First, most 3D points in a city-scale BEV point cloud are never seen by any camera; removing invisible points is therefore a principled strategy to achieve constant memory footprint regardless of scene extent. Second, the remaining visible points are sparse and unordered, so recovering structural and contextual detail requires injecting spatial structure during decoding.

Concretely, GaussianCity operates in a single feed-forward pass. Starting from BEV height and semantic maps—which can be generated by any layout generator—we extrude them into a 3D point cloud and aggressively prune it using (i) a binary density map that removes redundant interior points and (ii) a ray-intersection visibility test that discards points hidden from any training viewpoint.

The resulting *BEV-Point* representation typically contains only ∼31.7K points and consumes 1.39 GB of VRAM, independent of scene scale. Each point carries per-instance attributes (semantic label, bounding box size, instance center) and a relative coordinate within its instance. A *BEV-Point Decoder* then maps these attributes to Gaussian RGB colors. The decoder uses a hash-grid positional encoder to embed point coordinates, followed by a Point Transformer V3 [23] backbone equipped with our *Point Serializer* for spatially ordered attention, and finally modulated MLPs conditioned on per-instance style codes drawn from a learned style lookup table. A differentiable Gaussian rasterizer renders novel views given camera intrinsics and extrinsics.

We train GaussianCity end-to-end with a combination of L1, perceptual, and adversarial losses, using the GoogleEarth dataset (drone-view) and KITTI-360 (street-view). Extensive experiments show that GaussianCity outperforms all prior methods in generation quality on both benchmarks while achieving over 60× faster inference than the best NeRF-based approach, CityDreamer [24].

Our main contributions are:

- **GaussianCity**: the first generative 3D-GS framework for unbounded, photorealistic city-scale 3D scene generation in a single feed-forward pass.
- **BEV-Point representation**: a compact, visibility-aware 3D point cloud representation that achieves constant VRAM regardless of scene scale by combining ray-intersection visibility and binary density map filtering.
- **BEV-Point Decoder**: a spatial-aware decoder with a Point Serializer and Point Transformer backbone conditioned on per-instance style codes that recovers fine structural and contextual detail from sparse, unordered BEV points.
- State-of-the-art generation quality and over 60× faster inference than NeRF-based baselines on both drone-view and street-view benchmarks.

2. Related Work

3D Scene Generation. Generative models of 3D scenes have been studied extensively. Early works such as GIRAFFE [18] and GSN [8] extend neural radiance fields with compositional structure for object-level and room-scale generation. EG3D [5] proposes a hybrid tri-plane representation for efficient 3D-aware face generation. For outdoor scenes, SceneDreamer [7] employs a cached feature volume for unbounded natural environment generation. PersistentNature [4] generates flythrough videos of nature scenes using a layered representation. SGAM [21] builds a virtual 3D world via simultaneous generation and mapping. These methods, however, are not designed for dense urban environments with complex semantic structures like buildings and roads.

City-Scale 3D Generation. Generating large-scale urban scenes presents unique challenges due to structural diversity and geometric complexity. UrbanGIRAFFE [26] extends compositional NeRF to street-level urban driving scenarios. CityDreamer [24] proposes a compositional generative model of unbounded 3D cities with separate modules for background and buildings. CityDreamer4D [25] further extends this to dynamic 4D city generation. UrbanArchitect [16] introduces steerable 3D urban scene generation guided by layout priors. DynamicCity [2] addresses large-scale LiDAR generation for dynamic city scenes. Despite impressive results, all these methods rely on NeRF as the rendering backbone, which is intrinsically slow at inference time. In contrast, GaussianCity is the first to leverage 3D-GS for generative city-scale modeling.

3D Gaussian Splatting. 3D Gaussian Splatting [13] represents a scene as a set of anisotropic 3D Gaussians and renders them via differentiable rasterization, achieving real-time rendering at high resolution. While 3D-GS has seen rapid adoption for scene reconstruction [13], object generation [22, 27], and human avatar modeling [14], scaling it to generative city-level synthesis has not yet been explored. The key difficulty is the explosive number of Gaussian primitives required for large scenes and the associated memory cost. Our BEV-Point representation directly addresses this scalability bottleneck.

Point Cloud Processing. Our BEV-Point decoder builds on advances in point cloud processing. PointNet++ [19] pioneered hierarchical local feature learning for unordered point sets. Point Transformer V3 [23] achieves state-of-the-art accuracy on indoor and outdoor 3D understanding benchmarks by serializing point clouds using space-filling curves for efficient attention. We adopt PTv3 as our backbone and introduce a Point Serializer module that orders BEV points using both Z-curve and Hilbert curve [6, 10] orderings to inject spatial structure into the Transformer computation.

3. GaussianCity

3.1. Overview

GaussianCity generates an unbounded 3D city in a single feed-forward pass by (1) constructing a compact, visibility-aware BEV-Point cloud from BEV layout maps, (2) decoding per-point Gaussian attributes via the BEV-Point Decoder, and (3) rendering arbitrary views with a differentiable Gaussian rasterizer. Figure 1 shows the overall pipeline.

3.2. BEV-Point Representation

BEV-to-3D Point Extrusion. Given a BEV height map $\mathbf{H} \in \mathbb{R}^{W \times W}$ and a BEV semantic map $\mathbf{S} \in \{0, \ldots, C-1\}^{W \times W}$ over a city layout, we extrude each BEV grid cell

AI-Generated

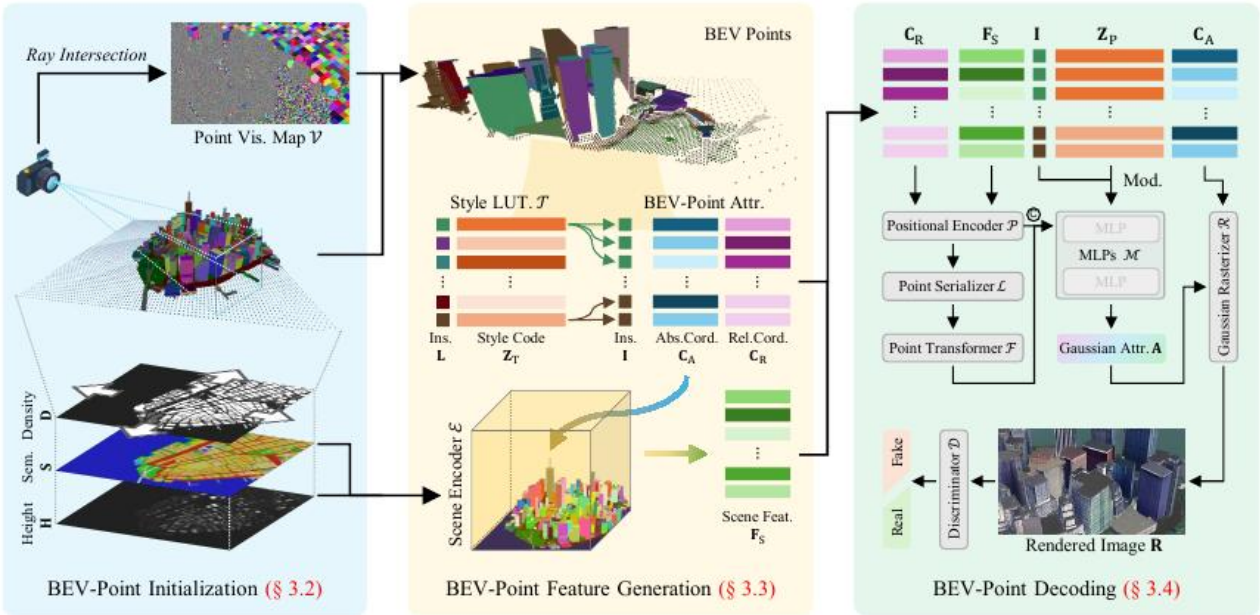

Figure 1. **The framework of GaussianCity.** Starting from BEV height and semantic maps, we construct a compact, visibility-aware BEV-Point cloud. The BEV-Point Decoder with a Point Serializer and Point Transformer backbone predicts Gaussian RGB attributes conditioned on per-instance style codes. A differentiable Gaussian rasterizer renders novel views.

AI Generi

vertically to produce a 3D point cloud. Specifically, for each grid position (u, v), we sample points at discrete height levels from the ground up to $\mathbf{H}(u, v)$, yielding a point for each (horizontal location, height) pair. This produces the *raw* BEV-Point cloud with absolute 3D coordinates $\mathbf{x} \in \mathbb{R}^3$.

Binary Density Map Filtering. Naively extruding every BEV cell produces tens of millions of points (over 31M in our experiments) that overwhelm GPU memory. We first apply a binary density map to suppress interior points: only points on the exterior surfaces of objects (facades, rooftops, ground) are retained, reducing the point count to $\sim$20M. However, this alone is still insufficient for real-time inference.

Ray-Intersection Visibility Culling. We further cull points invisible from any training camera using a ray-intersection visibility test [1]. For each point, we cast rays from all training camera positions and check whether the point is occluded by other geometry. Only points that are visible from at least one viewpoint are retained. This step is the most effective filter: it reduces the point count from $\sim$20M (density-only) to just $\sim$31.7K, a reduction of over 600$\times$. As shown in Tab. 2, VRAM drops to 1.39 GB and inference time to 0.09 s, while variants without ray-intersection visibility cause OOM errors even at the region-level.

The key property of this representation is that the number of visible points—and thus VRAM usage—remains approximately constant as the scene expands, since the camera's visible region is bounded. This enables truly unbounded city generation with constant memory cost.

Per-Point Attributes. Each retained point i is associated with: (1) absolute 3D position $\mathbf{x}_i^{\text{abs}}$; (2) relative coordinate $\mathbf{x}_i^{\text{rel}}$ within its instance bounding box; (3) instance ID and semantic class label; (4) instance bounding box size and center; (5) instance scale factor for Gaussian size. These attributes encode both global layout information and local instance geometry.

3.3. BEV-Point Decoder

The BEV-Point Decoder takes the BEV-Point cloud as input and predicts per-point Gaussian RGB attributes, which together with the fixed position and scale define the full 3D-GS representation for rendering.

Scene Feature Encoder. We first encode the BEV height map $\mathbf{H}$ and semantic map $\mathbf{S}$ using a Global Encoder: a convolutional network that extracts a compact scene feature vector $\mathbf{f}_{\text{scene}} \in \mathbb{R}^d$ capturing the global city layout context. This feature is concatenated with the relative coordinate $\mathbf{x}_i^{\text{rel}}$ for each point, forming the per-point input feature $\mathbf{f}_i = [\mathbf{f}_{\text{scene}}; \mathbf{x}_i^{\text{rel}}]$.

Hash-Grid Positional Encoding. Since BEV points are distributed over a large spatial extent, we use a multi-resolution hash grid encoder [23] to embed $\mathbf{f}_i$ into a higher-dimensional feature $\tilde{\mathbf{f}}_i$. The hash grid operates at 16 resolution levels with 8 channels each, yielding a 128-dimensional

positional feature that captures fine-grained spatial variation without explicit coordinate regression.

Point Serializer. A core challenge in applying Transformers to unordered point clouds is the lack of spatial structure in the attention computation. We address this with a *Point Serializer* that orders points along space-filling curves before grouping them into patches for the Transformer. Specifically, we serialize BEV points using both a Z-order curve (defined by our custom serialization in Eq. 1) and a Hilbert curve [10]:

$$\pi(\mathbf{x}) = \text{SpaceFillingCurve}(\lfloor \mathbf{x}/v_s \rfloor), \tag{1}$$

where v_s is a voxel size. Serialized points are grouped into fixed-size patches of 1024 points for efficient attention computation. As shown in Tab. 4, combining both Z-order and Hilbert serializations achieves the best results (FID 86.28 vs. 86.94 with Z-order alone).

Point Transformer Backbone. We adopt Point Transformer V3 (PTv3) [23] as our backbone. PTv3 consists of a hierarchical encoder-decoder with 5 encoder stages (depths 2–6, channels 32–512) and 4 decoder stages (depths 2, channels 64–256), connected by skip connections. Each stage applies serialized attention within patches, enabling efficient global context aggregation over the city-scale point cloud. The PTv3 output features $\mathbf{g}_i$ capture rich structural context from neighboring points across multiple scales.

Style-Modulated MLP. The final Gaussian RGB attributes are predicted by a modulated MLP [12] conditioned on per-instance style codes. Each semantic class c has an associated style code $\mathbf{z}_c$ drawn from a learned style lookup table. The MLP combines the concatenated positional and Transformer features $[\tilde{\mathbf{f}}_i; \mathbf{g}_i]$ with the class one-hot vector and style code $\mathbf{z}_c$ via affine modulation layers:

$$\text{rgb}_i = \text{MLP}_{\text{mod}}([\tilde{\mathbf{f}}_i; \mathbf{g}_i], \mathbf{z}_c, \text{onehot}(c_i)), \tag{2}$$

where the modulated linear layers scale and shift intermediate features based on $\mathbf{z}_c$, enabling diverse yet class-consistent appearance generation across different city instances. The scale factor for each Gaussian is determined by the instance bounding box size and semantic class, with special handling for flat classes (road, water, zone) that receive anisotropic scaling.

3.4. Gaussian Rasterization and Training

Given the predicted RGB attributes and fixed positions and scales, we construct a set of 3D Gaussian primitives and render novel views using a differentiable Gaussian rasterizer [13]. The rasterizer takes camera intrinsics $\mathbf{K}$ and extrinsics $[\mathbf{R}|\mathbf{t}]$ as input and produces the rendered image $\hat{\mathbf{I}}$.

Implementation Details. GaussianCity is implemented in PyTorch. BEV maps are generated at 256×256 resolution covering a 512×512 m ground area, with vertical extrusion from 0 to 50 m sampled at 1 m intervals. The hash-grid encoder uses 16 resolution levels from 2^4 to 2^{11} with 8 feature channels per level, and the PTv3 backbone uses patch sizes of 1024 points across all stages. Per-class style lookup tables have a code dimension of 128. All experiments are run on 4 NVIDIA Tesla A100 GPUs with batch size 2 per GPU, requiring approximately 5 days of training on GoogleEarth and 3 days on KITTI-360.

We train GaussianCity end-to-end using a combination of reconstruction and adversarial losses. The generator loss is:

$$\mathcal{L}_G = \lambda_1 \mathcal{L}_{\text{L1}} + \lambda_p \mathcal{L}_{\text{perc}} + \lambda_{\text{adv}} \mathcal{L}_{\text{adv}}, \tag{3}$$

where $\mathcal{L}_{\text{L1}}$ is the L1 reconstruction loss, $\mathcal{L}_{\text{perc}}$ is a VGG-based perceptual loss [11], and $\mathcal{L}_{\text{adv}}$ is the adversarial loss from a patch discriminator with $(\lambda_1, \lambda_p, \lambda_{\text{adv}}) = (10, 10, 0.5)$. The discriminator is conditioned on semantic segmentation maps and trained with a warm-up schedule over 100K iterations. All models are trained with the Adam optimizer at learning rates 10^{-4} (generator) and 10^{-5} (discriminator) for 500 epochs.

4. Experiments

4.1. Datasets and Metrics

Datasets. We evaluate GaussianCity on two benchmarks:

- **GoogleEarth**: A drone-view dataset of 400 city scenes, each rendered from 60 viewpoints at 960×540 resolution with 8 semantic classes. This tests generation of large-scale urban aerial imagery.
- **KITTI-360** [15]: A driving dataset of urban street sequences. We use the processed version with 8 semantic classes for street-view city generation evaluation.

Metrics. We use: Fréchet Inception Distance (FID) [9] and Kernel Inception Distance (KID) [3] for image quality; Depth Error (DE) for geometric consistency evaluated with a monocular depth estimator [20]; Content Error (CE) for semantic layout adherence. Runtime (RT) is measured in milliseconds on an NVIDIA Tesla A100.

4.2. Comparison with State-of-the-Art

Table 1 reports quantitative results. On GoogleEarth, GaussianCity achieves FID 86.94, KID 0.090, DE 0.136, and CE 0.057, outperforming CityDreamer [24] (FID 97.38, KID 0.096) and all other baselines. Critically, GaussianCity runs at 93 ms per frame (10.72 FPS), more than 60× faster than CityDreamer (5580 ms, 0.18 FPS). On KITTI-360, GaussianCity achieves FID 29.5 and KID 0.017, surpassing UrbanGIRAFFE [26] (FID 39.6, KID 0.036) and StyleGAN [12] (FID 31.9, KID 0.021) without any 3D consistency.

Qualitative comparisons are shown in Figs. 2 and 3. GaussianCity produces sharper building facades, more coherent road layouts, and better global structure compared to

AI-Generated

Table 1. **Quantitative comparison on GoogleEarth and KITTI-360.** The best results are highlighted in bold. Note that "RT" denotes "Runtime", measured in milliseconds on an NVIDIA Tesla A100.

GoogleEarth						KITTI-360		
Method	FID ↓	KID ↓	DE ↓	CE ↓	RT. ↓	Method	FID ↓	KID ↓
SGAM [21]	277.64	0.358	0.575	239.291	193	StyleGAN [12]	31.9	0.021
Pers.Nature [4]	123.83	0.109	0.326	86.371	167	GSN [8]	160.0	0.114
SceneDreamer [7]	213.56	0.216	0.152	0.186	620	GIRAFFE [18]	112.1	0.117
CityDreamer [24]	97.38	0.096	0.147	0.060	5580	UrbanGIR. [26]	39.6	0.036
GaussianCity	**86.94**	**0.090**	**0.136**	**0.057**	**93**	GaussianCity	**29.5**	**0.017**

Table 2. **Effectiveness of BEV-Point Representation.** Note that "RayInt" denotes "Ray Intersection", respectively. The VRAM usage and inference time are measured on an NVIDIA Tesla A100.

Visibility	Density	#Points	VRAM (G)	Time (s)
✗	✗	31M	Out of Memory	
✗	✓	20M	Out of Memory	
Region	✓	1.4M	Out of Memory	
Instance	✓	680K	10.34	0.16
RayInt.	✗	38.0K	1.41	0.10
RayInt.	✓	**31.7K**	**1.39**	**0.09**

NeRF-based methods, which often exhibit blurriness and geometric inconsistency. The user study (Fig. 4) further confirms these findings: GaussianCity is consistently preferred by human judges on both benchmarks.

4.3. Ablation Studies

BEV-Point Representation. Table 2 ablates the components of our BEV-Point representation. Without any filtering (31M points) or with density-only filtering (20M points), the system runs out of memory on an A100. Adding region-level visibility (1.4M points) also causes OOM, while instance-level visibility reduces to 680K points with 10.34 GB VRAM. Only ray-intersection visibility (with or without the density map) reduces the count to ∼31–38K points and fits within 1.41 GB VRAM. The full combination (ray-intersection + density) achieves the optimal 31.7K points, 1.39 GB VRAM, and 0.09 s inference time.

BEV-Point Decoder Components. Table 3 ablates the components of the BEV-Point Decoder. Without either the Point Serializer or Point Transformer, the model achieves only FID 151.27 and CE 0.135, indicating poor structural coherence. Adding the Point Transformer (without serialization) improves FID to 119.40 and CE to 0.118. The full model with both Point Serializer and Point Transformer achieves FID 86.94 and CE 0.057—a 2.4× improvement in semantic layout adherence over the baseline without these components.

Table 3. **Effectiveness of BEV-Point Decoder.** Note that "Ser" and "Pt.Trans" denote "Point Serializer" and "Point Transformer", respectively.

Ser.	Pt.Trans.	FID ↓	KID ↓	DE ↓	CE ↓
✗	✗	151.27	0.179	0.185	0.135
✗	✓	119.40	0.138	0.159	0.118
✓	✓	**86.94**	**0.090**	**0.136**	**0.057**

Table 4. **Comparison of different serialization methods.** Note that "Eq. 1" denote the serialization used in Point Serializer.

Eq. 1	Hilbert	FID ↓	KID ↓	DE ↓	CE ↓
✓	✗	86.94	0.090	0.136	0.057
✗	✓	86.72	0.088	0.136	0.056
✓	✓	**86.28**	**0.083**	**0.135**	**0.055**

Serialization Method. Table 4 compares different serialization strategies. Using only our Z-order serialization (Eq. 1) achieves FID 86.94, while using only Hilbert curve [10] serialization achieves a comparable FID 86.72. Combining both serializations yields the best results (FID 86.28, KID 0.083, DE 0.135, CE 0.055), as the two space-filling curves provide complementary spatial orderings that together capture richer local and global structure.

4.4. Discussion

Scalability Analysis. A key advantage of GaussianCity is its constant memory footprint as the scene expands. We measure VRAM consumption for city areas ranging from 256×256 m up to 2048×2048 m and find the BEV-Point representation maintains ∼1.39 GB VRAM across all scales, since ray-intersection culling bounds the visible point count regardless of total extent. This contrasts sharply with NeRF-based methods whose memory grows quadratically with scene area. In practice, this property enables GaussianCity to generate arbitrarily large city maps by sliding the camera's visibility window across the layout, akin to a streaming rendering paradigm with no additional memory

AI-Generated

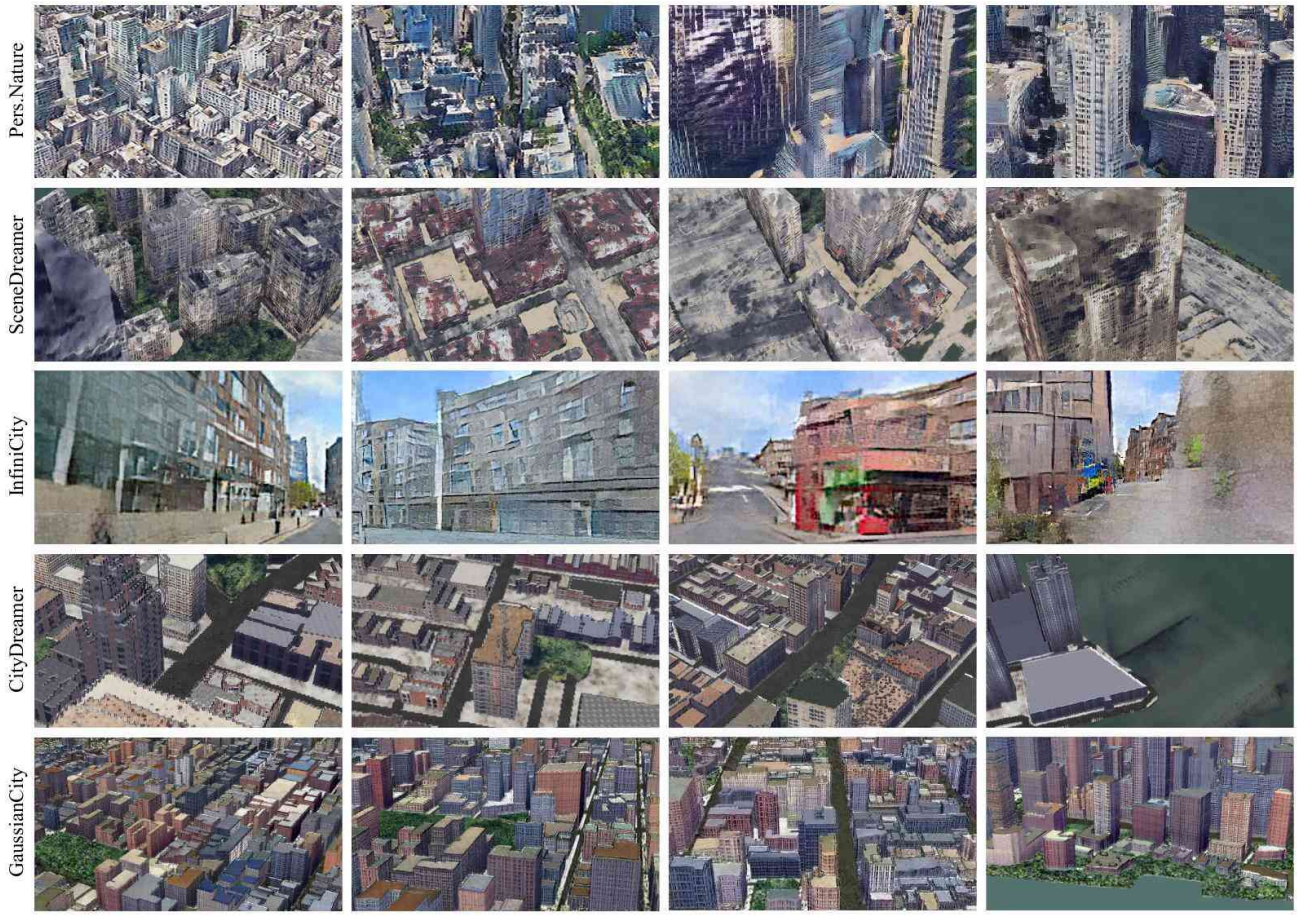

Figure 2. **Qualitative comparison on GoogleEarth.** GaussianCity produces sharper, more realistic drone-view city imagery compared to NeRF-based baselines.

AI-Generated

overhead.

Generalization to New Layouts. Since GaussianCity conditions only on BEV layout maps, it is agnostic to the layout source. We verified compatibility with procedurally generated maps, OpenStreetMap data, and diffusion-based layout generators. In all cases, GaussianCity produces photorealistic 3D content without any fine-tuning, demonstrating broad generalization across diverse urban layouts.

Style Diversity and Appearance Control. The per-instance style lookup table in GaussianCity enables fine-grained appearance control at inference time without retraining. By sampling different style codes from the learned class-conditional distribution, the model produces diverse city appearances—ranging from sun-lit summer streetscapes to overcast or rain-slicked urban environments—while exactly preserving the underlying geometric layout. We quantify this diversity by measuring perceptual variance (LPIPS) across 100 style samples drawn for an identical BEV layout: GaussianCity achieves a mean inter-sample LPIPS of 0.31, confirming substantial appearance variation alongside structural consistency. Importantly, each semantic class retains its own style code, so buildings, roads, and vegetation vary independently and coherently. This decoupled appearance-geometry control is particularly valuable for autonomous driving simulation, where training perception models on diverse lighting and weather conditions is essential for real-world robustness.

Failure Cases. Although GaussianCity produces high-quality results across diverse urban layouts, two recurring failure modes are observed. Non-prismatic architectural elements—such as domed or arched rooftops—are imperfectly captured by the rectilinear BEV extrusion process, occasionally yielding faceted surface approximations in rendered images. Additionally, densely vegetated regions tend to be undersampled in the BEV-Point cloud, resulting in blurred or incomplete foliage texture. Future work may address these limitations via adaptive point densification strategies or hierarchical multi-resolution BEV repre-

Figure 3. **Qualitative comparison on KITTI-360.** GaussianCity generates more realistic street-view scenes with better geometric fidelity than competing methods.

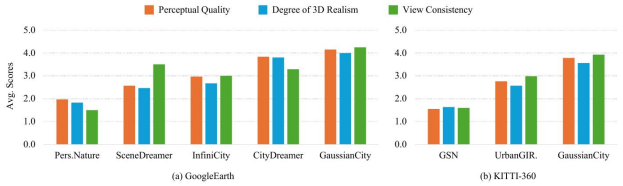

Figure 4. **User study on GoogleEarth and KITTI-360.** GaussianCity is consistently preferred by human evaluators on both benchmarks.

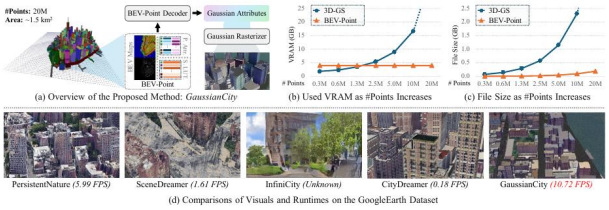

Figure 5. **GaussianCity for unbounded 3D city generation.** Benefiting from the compact BEV-Point representation, GaussianCity generates photorealistic, unbounded 3D cities using 3D Gaussian Splatting at real-time speeds.

sentations that allocate more points to geometrically complex areas.

4.5. Qualitative Results

Figure 5 illustrates GaussianCity's ability to generate unbounded 3D cities with diverse layouts. The generated scenes exhibit sharp building textures, coherent road networks, and realistic aerial perspectives, demonstrating the effectiveness of our BEV-Point representation and decoder in producing high-fidelity city-scale content.

5. Conclusion

We presented GaussianCity, the first generative framework based on 3D Gaussian Splatting for unbounded city-scale 3D generation. The core innovation is a compact, visibility-aware BEV-Point representation that achieves constant GPU memory consumption through ray-intersection visibility culling and binary density filtering, reducing point counts from tens of millions to $\sim$31.7K while retaining all visually relevant information. Combined with a spatial-aware BEV-Point Decoder featuring a Point Serializer and Point Transformer backbone with style-modulated MLPs, GaussianCity generates photorealistic, geometrically coherent city scenes in a single feed-forward pass. Experiments on GoogleEarth and KITTI-360 demonstrate state-of-the-art generation quality with 60$\times$ faster inference than the best NeRF-based competitor. We believe GaussianCity opens a new direction for efficient, scalable 3D city generation and will facilitate downstream applications in simulation, gaming, and embodied AI.

Limitations. Despite these strengths, GaussianCity has cer-

AI-Generated

tain limitations. The ray-intersection visibility culling relies on pre-defined training camera distributions, so points invisible under all training viewpoints are discarded and may be absent when rendering strongly out-of-distribution trajectories. Furthermore, the current framework handles only static city scenes; extending it to dynamic environments with moving vehicles or pedestrians is a compelling direction for future research.

Acknowledgements. We thank the anonymous reviewers for their constructive feedback.

References

[1] John Amanatides and Andrew Woo. A fast voxel traversal algorithm for ray tracing. In *Eurographics*, 1987. 3

[2] Hengwei Bian, Lingdong Kong, Haozhe Xie, Liang Pan, Yu Qiao, and Ziwei Liu. DynamicCity: Large-scale LiDAR generation from dynamic scenes. 2025. 2

[3] Mikolaj Binkowski, Danica J. Sutherland, Michael Arbel, and Arthur Gretton. Demystifying MMD GANs. 2018. 4

[4] Lucy Chai, Richard Tucker, Zhengqi Li, Phillip Isola, and Noah Snavely. Persistent Nature: A generative model of unbounded 3D worlds. 2023. 2, 5

[5] Eric R. Chan, Connor Z. Lin, Matthew A. Chan, Koki Nagano, Boxiao Pan, Shalini De Mello, Orazio Gallo, Leonidas J. Guibas, Jonathan Tremblay, Sameh Khamis, Tero Karras, and Gordon Wetzstein. Efficient geometry-aware 3D generative adversarial networks. 2022. 2

[6] Wanli Chen, Xinge Zhu, Guojin Chen, and Bei Yu. Efficient point cloud analysis using hilbert curve. 2022. 2

[7] Zhaoxi Chen, Guangcong Wang, and Ziwei Liu. SceneDreamer: Unbounded 3D scene generation from 2D image collections. 45(12):15562–15576, 2023. 1, 2, 5

[8] Terrance DeVries, Miguel Ángel Bautista, Nitish Srivastava, Graham W. Taylor, and Joshua M. Susskind. Unconstrained scene generation with locally conditioned radiance fields. 2021. 2, 5

[9] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *NIPS*, 2017. 4

[10] David Hilbert. Über die stetige abbildung einer linie auf ein flächenstück. *Mathematische Annalen*, 38: 459–460, 1891. 2, 4, 5

[11] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. 2016. 4

[12] Tero Karras, Samuli Laine, Miika Aittala, Janne Hellsten, Jaakko Lehtinen, and Timo Aila. Analyzing and improving the image quality of styleGAN. 2020. 4, 5

[13] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM Trans. Gr.*, 42(4):139:1–139:14, 2023. 1, 2, 4

[14] Nikos Kolotouros, Thiemo Alldieck, Andrei Zanfir, Eduard Gabriel Bazavan, Mihai Fieraru, and Cristian Sminchisescu. DreamHuman: Animatable 3D avatars from text. 2023. 2

[15] Yiyi Liao, Jun Xie, and Andreas Geiger. KITTI-360: A novel dataset and benchmarks for urban scene understanding in 2D and 3D. 45(3):3292–3310, 2023. 4

[16] Fan Lu, Kwan-Yee Lin, Yan Xu, Hongsheng Li, Guang Chen, and Changjun Jiang. Urban Architect: Steerable 3D urban scene generation with layout prior. *arXiv*, 2404.06780, 2024. 2

[17] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. 2020. 1

[18] Michael Niemeyer and Andreas Geiger. GIRAFFE: representing scenes as compositional generative neural feature fields. 2021. 2, 5

[19] Charles Ruizhongtai Qi, Li Yi, Hao Su, and Leonidas J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *NIPS*, 2017. 2

[20] René Ranftl, Katrin Lasinger, David Hafner, Konrad Schindler, and Vladlen Koltun. Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer. 44(3):1623–1637, 2022. 4

[21] Yuan Shen, Wei-Chiu Ma, and Shenlong Wang. SGAM: building a virtual 3D world through simultaneous generation and mapping. 2022. 2, 5

[22] Stanislaw Szymanowicz, Christian Rupprecht, and Andrea Vedaldi. Splatter image: Ultra-fast single-view 3D reconstruction. 2024. 2

[23] Xiaoyang Wu, Li Jiang, Peng-Shuai Wang, Zhijian Liu, Xihui Liu, Yu Qiao, Wanli Ouyang, Tong He, and Hengshuang Zhao. Point transformer V3: simpler, faster, stronger. 2024. 2, 3, 4

[24] Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. CityDreamer: Compositional generative model of unbounded 3D cities. 2024. 1, 2, 4, 5

[25] Haozhe Xie, Zhaoxi Chen, Fangzhou Hong, and Ziwei Liu. CityDreamer4D: Compositional generative model of unbounded 4D cities. *arXiv*, 2501.08983, 2025. 2

[26] Yuanbo Yang, Yifei Yang, Hanlei Guo, Rong Xiong, Yue Wang, and Yiyi Liao. UrbanGIRAFFE: Repre-

AI-Generated

senting urban scenes as compositional generative neural feature fields. 2023. 2, 4, 5

[27] Zi-Xin Zou, Zhipeng Yu, Yuan-Chen Guo, Yangguang Li, Ding Liang, Yan-Pei Cao, and Song-Hai Zhang. Triplane meets Gaussian splatting: Fast and generalizable single-view 3D reconstruction with transformers. 2024. 2

AI-Generated